

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Kinds in Haskell and Their Analogues in F# and JavaScript

Report

Course:

Advanced Programming

Student:

Claudio Bernardoni

617214

ACADEMIC YEAR 2025/2026

Contents

1 Kinds in Haskell **2**

1.1 Why a level above types is needed . . . 2
1.2 Kinds and their notation . . . 2
1.3 The kinds of type constructors and partial application . . . 3
1.4 Higher-kinded types . . . 3
1.5 Kind checking . . . 5
1.6 Type-level code . . . 6
1.7 Why kinds matter . . . 6

2 Kinds and the F# Type System **7**

2.1 Generic types in F# . . . 7
2.2 Generic type definitions as type constructors . . . 8
2.3 What the F# compiler checks instead of kinds . . . 9
2.4 Partial application of type constructors . . . 10
2.5 Higher-kinded polymorphism: the central limitation . . . 11
2.6 The Functor pattern without Functor . . . 12
2.7 Recovering part of the expressiveness . . . 13
2.8 Consequences for library design . . . 15
2.9 Summary: F# compared with Haskell's kinds . . . 15

3 Approximating Kinds in TypeScript **17**

3.1 TypeScript's types and JavaScript's run time . . . 17
3.2 Kinds in TypeScript: what exists natively . . . 18
3.3 A URI-based encoding of Functor . . . 19
3.4 What the encoding cannot express . . . 22
3.5 Practical value and comparison with F# . . . 23
3.6 Compilation and run time . . . 24

4 Conclusion **25**

1 Kinds in Haskell

Programmers are used to the idea that the type checker classifies *values* by their *types*. Less visible is the fact that the compiler also classifies *types* themselves. This second classification is the *kind system*. It is what allows abstractions such as `Functor` and `Monad` to be expressed and checked, and it is the concept against which F# and TypeScript will be compared in the following sections.

1.1 Why a level above types is needed

Consider a few values and their types:

```
True          :: Bool
Just 'x'      :: Maybe Char
[True, False] :: [Bool]
Left "oops"   :: Either String b
```

The expressions on the right-hand side are not all of the same nature. `Bool` and `Maybe Char` are types of values, but `Maybe` on its own is not: no value has type `Maybe`. `Maybe` is a *type constructor*, which must be applied to a type to *produce* a type. Likewise, `Either` needs two type arguments and the list constructor `[]` needs one; the notation `[Bool]` is simply syntax for the application `[] Bool`, and a function type `a -> b` is the application `(->) a b`.

The language of types therefore contains both complete types and *functions on types*, and with them comes the possibility of nonsense:

```
f :: Maybe -> Bool        -- Maybe is not a type of values
g :: Int Bool             -- Int cannot be applied to anything
```

To reject such expressions systematically, type expressions must themselves be classified according to how they may be used, just as types classify value expressions. These classifiers are called *kinds*. The result is the three-level picture of Table 1.

Table 1: Levels of classification in Haskell.

Level	Examples	Classified by
Values	`True`, `Just 3`, `not`	types
Types and type constructors	`Bool`, `Maybe Int`, `Maybe`, `Either`	kinds
Kinds	`Type`, `Type -> Type`	—[1]

1.2 Kinds and their notation

At their core, kinds are generated by this minimal grammar:

$$\kappa \;::=\; \mathsf{Type} \;\mid\; \kappa_1 \to \kappa_2$$

Type is the kind of ordinary types, those that have values, such as `Int` or `Maybe Char`. $\kappa_1 \to \kappa_2$ is the kind of a type constructor that takes a type of kind κ_1 and returns a type of kind κ_2. As with function types, the arrow associates to the right: $\mathsf{Type} \to \mathsf{Type} \to \mathsf{Type}$ means $\mathsf{Type} \to (\mathsf{Type} \to \mathsf{Type})$, so a two-parameter type constructor is a curried type-level function. In other words, kinds form a type-level language, one level above ordinary types.

[1] In standard Haskell the hierarchy stops here: kinds are not classified by anything the programmer can write. Modern GHC unifies types and kinds, so that kinds are themselves types and `Type` has kind `Type` (`Type :: Type`). This refinement is not needed in the rest of this report.

The kind of ordinary types was traditionally written $*$ ("star"). Modern Haskell uses the name `Type`, exported by the module `Data.Kind`, and this report writes `Type` throughout. The old notation can still be found in older code and in some compiler messages (for example `* -> *`); it denotes the same kind.

1.3 The kinds of type constructors and partial application

Asking the GHCi interpreter for the kinds of some familiar types and type constructors (with the command `:kind`) gives:

```
Int                :: Type
Maybe              :: Type -> Type
[]                 :: Type -> Type
Either             :: Type -> Type -> Type
Either String      :: Type -> Type
Either String Int  :: Type
(,) Bool           :: Type -> Type
(->) Int           :: Type -> Type
```

`Int` (like `Bool` or `Char`) has kind `Type`: it is a complete type. `Maybe` and `[]` have kind `Type -> Type`: given a type, they yield a type. `Either` has kind `Type -> Type -> Type`, as do the pair constructor `(,)` and the function arrow `(->)`.

Kinding rules. Kinds are assigned by two rules. Let Γ be the *kind environment*: it records the kind of every type constructor in scope (built in, like `[]` and `->`, or fixed by its declaration, like `Either` by `data Either a b = Left a | Right b`) and of every type variable in scope.

$$\frac{(T :: \kappa) \in \Gamma}{\Gamma \vdash T :: \kappa}\ \text{K-Con} \qquad \frac{\Gamma \vdash t_1 :: \kappa_1 \to \kappa_2 \qquad \Gamma \vdash t_2 :: \kappa_1}{\Gamma \vdash t_1\ t_2 :: \kappa_2}\ \text{K-App}$$

K-Con is the axiom: the kind of a name T (a type constructor or a type variable) is looked up, not derived. K-App, the type-level counterpart of the typing rule for function application, then computes the kind of every compound expression. For instance (omitting Γ, the leaves being K-Con look-ups):

$$\frac{\dfrac{\texttt{Either} :: \mathsf{Type} \to \mathsf{Type} \to \mathsf{Type} \qquad \texttt{String} :: \mathsf{Type}}{\texttt{Either String} :: \mathsf{Type} \to \mathsf{Type}}\ \text{K-App} \qquad \texttt{Int} :: \mathsf{Type}}{\texttt{Either String Int} :: \mathsf{Type}}\ \text{K-App}$$

Partial application. Because kinds are curried, a type constructor can be applied to fewer arguments than it ultimately needs, and each argument removes one arrow from the front of its kind: `Either` needs two more arguments, `Either String` one, and `Either String Int` none. A partially applied constructor such as `Either String`, `(,) Bool` or `(->) Int` is not a type of values, but it is a perfectly valid type expression with the same kind, `Type -> Type`, as `Maybe` or `[]`. As the next subsection shows, it can therefore be used wherever a one-parameter type constructor is expected.

Partial application always fixes the *leading* arguments: Haskell has no type-level lambda, so there is no direct way to write "`Either` with its *second* argument fixed". This has visible consequences, as we shall see with `Functor`.

1.4 Higher-kinded types

In all the kinds seen so far, every argument has kind `Type`. The grammar of kinds, however, also allows arguments of arrow kind, as in `(Type -> Type) -> Type`. We use the following

terminology: a *higher-kinded type variable* ranges over type constructors rather than over types (typically, its kind is `Type -> Type`); a *higher-kinded type* is parameterised by such a variable; and *higher-kinded polymorphism* is the ability to write definitions that are polymorphic in such a variable. The term "higher-kinded type" is also often used loosely for any type constructor of arrow kind, such as `Maybe`; what really distinguishes languages is whether one can *abstract over* such constructors.

`Functor`. The canonical example is the `Functor` class, shown here with its kind written explicitly:

```
class Functor (f :: Type -> Type) where
  fmap :: (a -> b) -> f a -> f b
```

The parameter `f` is not a type but a type constructor: in the type of `fmap` it is applied to `a` and to `b`. Its instances include `Maybe`, `[]` and `IO`, but also the partially applied `Either e` and `((,) a)`, all of kind `Type -> Type`. Because partial application fixes only leading arguments, `fmap` on `Either e` can map over the `Right` component but never over the `Left` one. Classes of this sort, which range over type constructors rather than types, are known as *constructor classes*.

Higher-kinded polymorphism lets a single definition work for every such container:

```
double :: Functor f => f Int -> f Int
double = fmap (* 2)

-- double (Just 21)                        == Just 42
-- double [1, 2, 3]                        == [2, 4, 6]
-- double (Right 5 :: Either String Int)   == Right 10
```

Here `f` is instantiated to `Maybe`, `[]` and `Either String` respectively. Without a variable of kind `Type -> Type`, a separate function would be needed for each container. The part `Functor f =>` of the signature is a *constraint*: it restricts `f` to the type constructors that are instances of `Functor`. Constraints are classified by kinds too: `Functor f` has the kind `Constraint` (also exported by `Data.Kind`), so the class itself behaves as a type-level function `Functor :: (Type -> Type) -> Constraint`, and writing `Functor Int` is rejected for the same reason as any ill-kinded application.

Higher-kinded data types. Data types can take type constructors as parameters too. The following tree is parameterised by the container that holds the children of each node:

```
data Rose f a = RNode a (f (Rose f a))
-- Rose :: (Type -> Type) -> Type -> Type
-- Rose []    Int : each node has a list of children
-- Rose Maybe Int : each node has at most one child

example :: Rose [] Int
example = RNode 1 [RNode 2 [], RNode 3 []]
```

Unlike the `Tree` type of the next subsection, `Rose` has no separate constructor for leaves, and it does not need one: a leaf is a node whose container of children is empty. With `f = []` a leaf is `RNode x []`, as in `example`; with `f = Maybe` it is `RNode x Nothing`. The base case is thus supplied by the container `f`, because `f` must be a type constructor that can be applied to `Rose f a`.

The same idea scales to library design. A *monad* is, for our purposes, a type constructor `m :: Type -> Type` whose values `m a` describe computations producing an `a`, possibly with

an effect: `Maybe` for computations that may fail, `IO` for input and output. A *monad transformer* adds an effect to an arbitrary monad. For example, `newtype MaybeT m a = MaybeT (m (Maybe a))` adds the possibility of failure to any monad `m`. Since it takes a monad and returns a new one, `MaybeT` has kind `(Type -> Type) -> Type -> Type`, the same kind as `Rose`, and the class of all monad transformers ranges over constructors of that kind.

1.5 Kind checking

The compiler uses kinds to verify that every type expression in a program is well formed, in the same way as it uses types to verify value expressions. Three checks do most of the work:

1. **Types of values must have kind `Type`.** Any type that classifies values (in a type signature, a constructor field or an argument of `->`) must have kind `Type`.
2. **Applications must be well kinded.** Every type application must satisfy K-App.
3. **Instances must match their class.** Every instance of a class must have the kind of the class parameter; for `Functor`, `Type -> Type`.

A valid example. In the following code, `Tree` has kind `Type -> Type`, which is exactly the kind required of the parameter of `Functor`:

```
data Tree a = Leaf | Node (Tree a) a (Tree a)    -- Tree :: Type -> Type

instance Functor Tree where                      -- f := Tree
  fmap _ Leaf         = Leaf
  fmap g (Node l x r) = Node (fmap g l) (g x) (fmap g r)
```

Kind errors. Now consider three ill-kinded declarations, one for each check:

```
isNothing :: Maybe -> Bool          -- violates check 1
isNothing Nothing = True
isNothing _       = False

g :: Int Bool                       -- violates check 2

instance Functor Int where          -- violates check 3
  fmap = undefined
```

In the first, the arrow requires an argument of kind `Type`, but `Maybe` has kind `Type -> Type`. GHC reports:

```
    * Expecting one more argument to 'Maybe'
      Expected a type, but 'Maybe' has kind '* -> *'
```

The fix is to supply the missing argument: `Maybe a -> Bool`. In the second, K-App requires `Int` to have an arrow kind, but `Int :: Type` cannot be applied to anything. In the third, `Functor` expects a constructor of kind `Type -> Type` but `Int` has kind `Type`, so no sensible `fmap` could even be typed. In all three cases the error is detected from kinds alone, at the declaration where it occurs, without examining any value-level code.

Kind inference. Kinds rarely have to be written: the compiler infers them by unification, much as it infers types. For the rose tree above, write κ_f and κ_a for the unknown kinds of its parameters. A data type always has result kind `Type`, so `Rose` $:: \kappa_f \to \kappa_a \to$ `Type`.

The field `a` is the type of a value, so $\kappa_a = \mathsf{Type}$. In the field `f (Rose f a)`, `f` is applied to an argument of kind `Type` and yields the type of a value, so $\kappa_f = \mathsf{Type} \to \mathsf{Type}$. Hence `Rose` :: $(\mathsf{Type} \to \mathsf{Type}) \to \mathsf{Type} \to \mathsf{Type}$. When the constraints leave a kind undetermined, standard Haskell defaults it to `Type`, while GHC by default generalises it to a polymorphic kind.

1.6 Type-level code

In Haskell the language of types is itself a small programming language, whose "programs" are evaluated by the compiler rather than at run time. By *type-level code* we mean any construct that builds or computes types during compilation: type constructor application (the analogue of function application), *type synonyms*, i.e. new names for existing types declared with `type`, which may take parameters (`type Pair a = (a, a)` makes `Pair Int` an abbreviation of `(Int, Int)`), and, in GHC, *type families*, which are functions from types to types defined by equations:

```
type family Elem (c :: Type) :: Type where   -- Elem :: Type -> Type
  Elem [e]       = e
  Elem (Maybe e) = e
```

If types form a programming language, kinds are its type system. Before any type-level expression is used or reduced, the compiler checks that each application has arguments of the expected kinds and that each result has the kind required by its context. Thus `Elem [Bool]` is well kinded and reduces to `Bool`, whereas `Elem Maybe` is rejected before any reduction is attempted, because `Maybe` has kind `Type -> Type` rather than `Type`. Kind checking plays, one level up, exactly the role that type checking plays for ordinary code. GHC extensions such as `DataKinds` go further and let programmers define new kinds, so that kind checking can enforce more precise invariants, for example that a vector's length index is a natural number rather than an arbitrary type.

1.7 Why kinds matter

Kinds are more than a formal classification. In practice they provide:

- **Safe abstraction over type constructors.** `Functor`, `Monad` and monad transformers all depend on variables of kind `Type -> Type` or higher; writing `double` once for all containers is the direct payoff.
- **Early, local error detection.** Misuses of type constructors are reported at the offending declaration, with messages phrased in terms of kinds, as in the `isNothing` example.
- **Documentation of interfaces.** A kind annotation such as `(f :: Type -> Type)` states the shape expected of a type parameter at a glance.
- **An explanation of design constraints.** Kinds explain, for instance, why `Functor` instances exist for `Either e` but can only map over the last type parameter.
- **A foundation for type-level programming.** Kind checking keeps type-level code well formed, just as type checking does for value-level code.

Finally, kinds give a precise criterion for comparing languages: can a language express a variable of kind `Type -> Type`? This is the question that guides the discussion of F# and of TypeScript in the following sections.

2 Kinds and the F# Type System

Section 1 closed with a test question: can a language express a variable of kind `Type -> Type`? This section applies the test to F#, a functional-first language of the ML family that runs on .NET, and asks more generally:

> *To what extent can F# express and reason about the same abstractions that Haskell expresses through kinds and higher-kinded types?*

The answer, developed below, has three parts. F# has generic type definitions that correspond to Haskell's *first-order* type constructors used fully applied, and its compiler rejects ill-formed applications of them. It does not, however, have a kind system in the sense of Section 1: there are no arrow kinds, a type constructor is never a type expression on its own, and no type variable can range over type constructors. Consequently F# has no direct counterpart of `Functor`, and the workarounds used in practice each recover only part of what Haskell's kinds provide.

A note on terminology. In F#'s own vocabulary, the "kind" of a type definition is whether it is a class, an interface, a record, a union, and so on. That usage is unrelated to kinds as classifiers of type expressions; in this section "kind" always has the meaning of Section 1.

2.1 Generic types in F#

Only three features of the F# type system matter for the comparison.

Generic type definitions. A type definition may declare *generic type parameters*, written with a leading apostrophe (`'T`, `'TError`). The core library declares the counterparts of `Maybe`, `[]` and `Either` as follows:

```haskell
-- Haskell
data Maybe a
  = Nothing
  | Just a
data [] a       -- built in:
  = []          -- illustration
  | a : [a]     -- only
data Either a b
  = Left a
  | Right b
```

```fsharp
// F#
type Option<'T> =
    | None
    | Some of 'T
type List<'T> =
    | ([])
    | (::) of 'T * 'T list
type Result<'T, 'TError> =
    | Ok of 'T
    | Error of 'TError
```

The Haskell list type is built-in syntax that no module may redeclare, so its declaration is only illustrative; the F# declarations are the core library's, without attributes and field names. F# code usually writes `option<'T>` and `list<'T>`: the core library defines these lower-case names, as in ML, as type abbreviations (aliases) of `Option<'T>` and `List<'T>`.

The parameter order of `Result` also differs from that of `Either`: `Result<'T,'TError>` takes the success type first. `Either` itself is neutral, but by convention Haskell code puts the successful result in `Right`, which matches the `Functor` and `Monad` instances of `Either e`: they can only act on the last parameter (Section 1.4). `Result` is bound by no such instance (Section 2.6).

Type application. A generic type definition is used by supplying type arguments in angle brackets, `option<int>` or `Result<int, string>`; a one-parameter type may also be applied in postfix (ML-style) notation, as in `int option` or `int list`, which means exactly the same as `option<int>`.

Inference and constraints. F# infers types in the Hindley–Milner tradition and generalises `let`-bound functions automatically. For instance,

```
let swap (x, y) = (y, x)
    // inferred: 'a * 'b -> 'b * 'a
let sortedPair x y = if x < y then (x, y) else (y, x)
    // inferred: 'a -> 'a -> 'a * 'a when 'a : comparison
```

The second function is generic but *constrained*: the use of `<` forces its type parameter to satisfy `comparison`. Constraints can also be declared explicitly with `when`. Each form is a condition on the type that instantiates the parameter: `'T :> IDisposable` requires that type to be `IDisposable` or to implement it; `'T : equality` and `'T : comparison` require it to support equality or ordering; `'T : struct` and `'T : not struct` require a .NET value type or a reference type; other forms require a parameterless constructor or support for `null`; *member constraints* are discussed in Section 2.7. Whatever their form, constraints are predicates on a type parameter of kind `Type`: they narrow down *which* types may be supplied, never *what sort* of thing (a type, not a type constructor) is supplied.

Finally, generic type definitions are compiled to .NET generic types, which exist at run time with their number of parameters recorded in their name: `Option<'T>`, `List<'T>` and `Result<'T,'TError>` are compiled to the .NET types `FSharpOption`1`, `FSharpList`1` and `FSharpResult`2`.

2.2 Generic type definitions as type constructors

Seen from outside, `option` behaves as `Maybe` does: given the type `int`, it produces the type `option<int>`, and no value has type "`option`" alone. It is therefore natural to describe the F# definitions with Haskell's kinds, as in Table 2.

Table 2: Haskell type constructors and their F# counterparts.

Haskell	Kind in Haskell	F# definition	Parameters
`Maybe`	`Type -> Type`	`option<'T>`	1
`[]`	`Type -> Type`	`list<'T>`	1
`Either`	`Type -> Type -> Type`	`Result<'T,'TError>`	2

The description is accurate, but it is a description *about* F#, not something the language can state. Two differences matter.

Generic definitions are always used saturated. In Haskell, `Maybe` on its own is a well-formed type expression of kind `Type -> Type`; it can be passed as an argument, as in `Rose Maybe Int`. In F#, `option` without arguments is not a type expression: a generic type definition may appear in a type only when applied to *exactly* as many arguments as it declares. The wildcard `_` does not change this. In `let f (x: option<_>) = x` the underscore is an anonymous type variable to be inferred, so `option` is still fully applied.

Application is uncurried. A two-parameter definition such as `Result<'T,'TError>` receives its arguments together; there is no intermediate type "`Result<int>`", with only its first argument supplied, comparable to `Either Int :: Type -> Type`. A closer description of `Result` would therefore be "a constructor of arity 2" rather than Type → Type → Type. Arity is even part of a definition's identity: the core library declares both a generic type `Async<'T>` and a non-generic type `Async` (the class of operations such as `Async.RunSynchronously`) in the same namespace, and the number of type arguments written selects which one is meant;

.NET types such as `Action` and `Action<'T>` work the same way. In Haskell, by contrast, a module cannot declare two type constructors with the same name at all, so a name determines a single constructor, and with it a single kind.

In Haskell's terms, then, every F# type constructor has a first-order kind $\mathsf{Type} \to \cdots \to \mathsf{Type} \to \mathsf{Type}$ (with n arguments) and is only ever used fully applied, while every F# type variable has kind Type (measure parameters aside; see Section 2.3). Since nothing else can occur, the arity of each definition (with the sort of each parameter) carries all the information that kinds carry in this fragment, which is why F# never needs to talk about kinds.

2.3 What the F# compiler checks instead of kinds

Apart from the `[<Measure>]` attribute discussed below, F# programs contain nothing resembling kind annotations, the compiler infers no arrow kinds, and its messages speak of type arguments, not kinds: F# has no explicit kind system comparable to Haskell's. The compiler nevertheless checks type expressions in several ways, some with the same practical effect as parts of Haskell's kind checking.

1. **Generic arity.** A named type must be given as many type arguments as its definition declares, and a non-generic type none at all. This covers the combination of K-App and the requirement of Section 1.5 that types of values have kind `Type`.

2. **Only named types can be applied.** The head of a type application must name a type definition (or abbreviation). A type variable cannot be applied to arguments: `'F<'A>` is parsed, but the type checker rejects it with the error "*Type parameter cannot be used as type constructor*" (FS0712). This is the restriction that rules out higher-kinded polymorphism, discussed in Section 2.5.

3. **Constraints of the parameters.** Each type argument must satisfy the constraints declared for its parameter. The core library declares `Set<'T when 'T : comparison>`, so `Set<int -> int>` is rejected because function types do not support comparison. A type abbreviation must even restate the constraints of the type it abbreviates.

4. **Types versus units of measure.** F# type parameters come in two sorts: ordinary type parameters, and *measure* parameters, declared with the attribute `[<Measure>]`, which range over units such as metres or seconds. The core library defines, for instance, `float<[<Measure>] 'Measure>`. The compiler assigns one of the two sorts to every parameter and rejects `float<int>` (a type where a unit is expected) as well as `list<m>` (a unit where a type is expected).

5. **Well-formed abbreviations.** The expansion of type abbreviations must terminate, and an abbreviation must use all of its declared type parameters on its right-hand side.

Each of the following declarations violates the check indicated in its comment, and each is rejected at the declaration, before any value-level code is examined:

```
[<Measure>] type m                        // a unit of measure

// Check 1: generic arity
let isNone (x: option) = x.IsNone         // FS0033: expects 1 type argument, given 0
let g (x: string<bool>) = x               // string is not a generic type
// Check 2: only named types are applied
type Rose<'F, 'A> = Node of 'A * 'F<Rose<'F, 'A>>    // FS0712
// Check 3: constraints
let s : Set<int -> int> = Set.empty       // no comparison on int -> int
// Check 4: types versus units of measure
```

```
let h (x: float<int>) = x                 // FS0705: unit of measure expected
let l : list<m> = []                      // FS0704: type expected
// Check 5: well-formed abbreviations
type Loop = option<Loop>                  // FS0953: cyclic abbreviation
type Const<'A, 'B> = 'A                   // 'B unused (Section 2.4)
```

Only the first two checks describe kind checking in the sense of Section 1, and in F# they reduce to counting arguments and forbidding applied type variables: FS0712 is exactly the failure of K-App when every type variable has kind `Type`. Check 3 is closer to a Haskell *class* constraint than to a kind: constraints single out a subset of the types of kind `Type` (those with comparison, those with a parameterless constructor, ...), whereas kinds classify type expressions by their *shape*. Check 4 is the one place where F# classifies type-level entities by sort, and the compiler's internal representation indeed calls this distinction the kind of a type parameter. It is, however, a kind-like distinction between two base sorts, *type* and *measure*, with no arrows: it separates units from types, but it cannot describe a parameter that is itself a type constructor. Check 5 concerns abbreviations, which Haskell also restricts (cyclic synonyms are rejected there too), though not in exactly the same way (Section 2.4).

Type constructors at run time. An unapplied generic type definition is a first-class entity only in the .NET reflection API, outside the static type system: `typedefof<int list>` returns the definition `FSharpList`1` as a `System.Type` value, and `MakeGenericType` applies it to type arguments, throwing an exception at run time if their number is wrong or a constraint is violated: arity and constraints again.

2.4 Partial application of type constructors

In Haskell, `Either String` is a type expression of kind `Type -> Type`, and it can instantiate a type variable `f` of that kind (Section 1.4). The closest F# construct is a type abbreviation with a parameter:

```
-- Haskell
type Validated = Either String
-- Validated :: Type -> Type

v :: Validated Int
v = Right 5

w :: Validated Int
w = fmap (+ 1) v
-- f := Either String
```

```
// F#
type Validated<'T> = Result<'T, string>

let v : Validated<int> = Ok 5

let w = Result.map ((+) 1) v
// w : Result<int, string>, which is
// the same type as Validated<int>

// No unapplied form exists. Rejected:
//   type V1 = Result              FS0033
//   type V2 = Result<_, string>   FS0715
```

The abbreviation looks like a partial application, but its Haskell counterpart is a different construct: a type synonym *with a parameter*, such as `type Checked a = Either a String`. In both languages such a synonym is a named type-level function, `Validated` $= \lambda T.$ `Result`$\langle T, \texttt{string}\rangle$, so it may fix any argument (here the second), and in both languages it must be applied to all of its declared parameters wherever it is used. The F# abbreviation is therefore not equivalent to Haskell's partial application, for three reasons.

1. **It is not a type expression on its own.** Like any generic definition, `Validated` must be applied to all of its parameters wherever it is used. A Haskell synonym can avoid this by being declared with *fewer* parameters: the synonym `type Validated = Either String` above has none, so it is saturated when written alone and denotes a

constructor of kind `Type -> Type`, usable as in `Functor Validated`.[2] F# has no such form, because the right-hand side of an abbreviation must be a type: `type V1 = Result` lacks type arguments, and in `type V2 = Result<_, string>` the underscore stands for an anonymous type variable to be inferred, not for an abstracted argument, and such variables are not allowed in type definitions.

2. **It is erased.** Abbreviations are expanded during checking and do not exist in compiled code; `Validated<int>` *is* `Result<int, string>`. The abbreviation is a syntactic convenience, not a new type, so it cannot carry members or interface implementations of its own.

3. **There is nothing for it to instantiate.** The point of `Either String :: Type -> Type` is that it can be the argument of something expecting a type constructor, as `f` in `Functor f`. In F# no type parameter can stand for a type constructor, so even a perfect partial application would have nowhere to go. Accordingly, `w` above calls `Result.map` directly instead of a generic `map`.

What F# lacks is any type expression of arrow kind: neither unsaturated type constructors such as `Either String`, nor synonyms that denote them. Abbreviations are also more restricted than synonyms in one detail: they must use every parameter they declare, so a "constant" abbreviation such as `type Const<'A,'B> = 'A` is rejected, whereas `type Const a b = a` is legal Haskell.

2.5 Higher-kinded polymorphism: the central limitation

We can now ask the central question directly. Can F# declare a parameter corresponding to `f :: Type -> Type` and use it as `fmap :: (a -> b) -> f a -> f b` does? The literal transcription of `Functor` into an F# interface is:

```
type Functor<'F> =
    abstract Map<'A, 'B> : ('A -> 'B) -> 'F<'A> -> 'F<'B>
    // error FS0712: Type parameter cannot be used as type constructor
```

The declaration of `'F` is accepted, but `'F` is an ordinary type parameter, implicitly of kind `Type`, and the attempt to apply it to `'A` is rejected. No syntax gives a type parameter an arrow kind, and the type checker accepts only named types at the head of an application. F# therefore cannot express:

- a type variable ranging over type constructors (`f` in `Functor f`, `m` in `Monad m`);
- a generic type with such a parameter, such as `Rose :: (Type -> Type) -> Type -> Type` or the monad transformer `MaybeT` of Section 1.4;
- a constraint or interface over type constructors, i.e. any counterpart of a constructor class.

The .NET runtime has the same restriction: its generic parameters can only be instantiated with types. Anything polymorphic only in the element type, such as `List.map`, remains expressible.

[2] GHCi's `:kind` command is more permissive than the language: for `type Either2 a b = Either a b` it accepts `:kind Either2 Int` and reports `Type -> Type`, although a program that uses `Either2 Int` unsaturated, e.g. in `Functor (Either2 Int)`, is rejected because the synonym `Either2` should have 2 arguments.

Why passing `map` as an argument is not enough. One might hope to avoid `'F<'A>` by passing the mapping function of the container as an ordinary argument, in the style of explicit dictionary passing. This works for `double` of Section 1.4, because there the element type never changes:

```
let double map xs = map (fun n -> n * 2) xs
double List.map [1; 2; 3]                    // [2; 4; 6]
double Option.map (Some 21)                  // Some 42
```

It fails as soon as the mapping function is needed at two different element types:

```
-- Haskell: accepted
twice :: Functor f
      => f Int -> f String
twice xs =
  fmap show (fmap (* 2) xs)
```

```
// F#: rejected (type mismatch)
let twice map xs =
    map string (map (fun n -> n * 2) xs)
```

In F#, as in any Hindley–Milner language, generalisation happens only at definitions (`let` and members), so the parameter `map` has a single, monomorphic type inside the body. Its first use fixes it to `(int -> int) -> ...`, and the second use, at `int -> string`, does not match. In Haskell, `Functor f =>` supplies an `fmap` that is still polymorphic in `a` and `b` while `f` is fixed. Passing such an argument explicitly would need two things. First, the argument must stay polymorphic: its type would be `forall a b. (a -> b) -> f a -> f b`, with the quantifier *inside* the type of a parameter (a rank-2 type). An F# function parameter cannot have such a type, but a generic method of an object can, and Section 2.7 exploits this. Second, that type applies a variable `f` to the varying element types, and this is what F# cannot express in any form. Higher-kinded polymorphism is thus not a luxury in Haskell's `Functor`: it is what allows "the same container, different element types" to be written as a type.

2.6 The Functor pattern without Functor

The F# core library offers a separate `map` for each container, in the module named after the type:

```
List.map   : ('T -> 'U) -> 'T list -> 'U list
Option.map : ('T -> 'U) -> 'T option -> 'U option
Result.map : ('T -> 'U) -> Result<'T,'TError> -> Result<'U,'TError>
Array.map  : ('T -> 'U) -> 'T[] -> 'U[]
```

All four signatures are instances of one pattern, `(a -> b) -> f a -> f b`, with `f` instantiated to `list`, `option`, "`Result` with its error type fixed" and arrays. In Haskell this pattern *is* a type, the type of `fmap`, and the four functions would be four instances of one class. In F# the pattern exists only in the reader's mind and in naming conventions: no F# type mentions all four, and each function is generic only in its element types `'T` and `'U`. When F# programmers need to speak about such patterns, for instance to describe the methods of computation expression builders (below), they resort to an informal notation `M<'T>` that the language itself cannot express.

Two consequences follow:

- **Generic code must be repeated or specialised.** A Haskell function such as `twice` is written once for every functor. In F# it is written once per container (`List.map string (List.map (fun n -> n * 2) xs)`, and so on), or made generic over the element type only.
- **No kind dictates which parameter is mapped.** Haskell's `Functor (Either e)` can

only map over the last parameter of `Either`, because partial application fixes leading arguments (Section 1.4). F#'s `Result.map` maps over the *first* parameter, the success value, and the library also provides `Result.mapError` for the second. With no constructor class to satisfy, each module function simply chooses the parameter it transforms.

The first point motivates the workarounds that follow.

2.7 Recovering part of the expressiveness

A Haskell constructor class such as `Functor` provides three things at once: *overloading* (one name `fmap` for every container), *abstraction* (functions such as `twice` written once and compiled once for every container) and *kind discipline* (a static guarantee that the instance is a type constructor of kind `Type -> Type`). Each F# technique below recovers some of these and gives up others.

Inline functions and member constraints. F# allows a function marked `inline` to use *statically resolved type parameters*, written `^T` (or, since F# 7, also `'T`), together with *member constraints* which require a type to have a member of a given name and signature. Because the function is inlined at every call site, the constraint is resolved at compile time against the concrete type found there; no .NET generic code needs to represent it. This approximates the *overloading* part of `Functor`.

```
type Box<'T> =
    | Box of 'T
    static member Map (b: Box<'T>, f: 'T -> 'U) : Box<'U> =
        let (Box x) = b in Box (f x)

let inline fmap (f: 'T -> 'U) (x: ^Fa) : ^Fb =
    (^Fa : (static member Map : ^Fa * ('T -> 'U) -> ^Fb) (x, f))

fmap (fun n -> n * 2) (Box 21)                // Box 42
```

The signature of `fmap` is instructive. Its input and output types are two *unrelated* type variables `^Fa` and `^Fb`: nothing says that they are the same constructor applied to `'T` and `'U`, because F# cannot say it; the relationship holds only because each type's `Map` member happens to have that shape. The best-known library built on this technique, FSharpPlus, makes the point visible by naming its type variables ``` ``Functor<'T>`` ``` and ``` ``Functor<'U>`` ```: these are single type variables whose *names* contain angle brackets, not applications. The library also has to work around the fact that `list`, `option` and other existing types do not declare a static `Map` member: it collects one `Map` overload per supported type in a helper class, and lets overload resolution at each inlined call site pick the right one. The limitations are significant:

- **No kind discipline.** Since `^Fa` has kind `Type`, any type with a suitable member qualifies. FSharpPlus's `map` accepts a `string` (mapping over its characters) which Haskell's kinds would reject as `Functor` instances (`String :: Type`).
- **Inline only.** Statically resolved type parameters can only be generalised in `inline` definitions, and cannot be used on type definitions. There is no separately compiled generic code, no interface or data type over "any functor" (no counterpart of `Rose f a`), and a function such as `fmap` cannot be passed as a first-class value without fixing its types.
- **Closed or ad hoc instances.** A member constraint is satisfied only by members defined with the type or in its intrinsic extensions; to support existing types, the overloads must be listed centrally, as FSharpPlus does.

Since F# 7, a requirement such as "has a static member `Add`" can also be stated by name, as an *interface with static abstract members*:

```
type IAddable<'T when 'T :> IAddable<'T>> =
    static abstract Add : 'T * 'T -> 'T

let addSelf<'T when 'T :> IAddable<'T>> (x: 'T) = 'T.Add(x, x)   // not inline
```

Unlike a member constraint, this requirement has a name, a type meets it only by declaring that it implements `IAddable`, and functions using it need not be `inline`.

Explicit witnesses and an encoding of type application. The *abstraction* part of `Functor`, generic code compiled once, can be recovered by representing the forbidden application `'F<'A>` as an ordinary two-parameter type. This technique, known as *lightweight higher-kinded polymorphism*, was introduced for OCaml, which has the same restriction. A generic type `App<'F,'A>` stands for "`'F` applied to `'A`", and each container gets an uninhabited *brand* type that names it, together with an injection and a projection between the real type and its encoded form:

```
type App<'F, 'A> = interface end              // encodes the application 'F<'A>

type ListBrand = class end                    // a name for the constructor list
type ListApp<'A>(xs: 'A list) =               // should be hidden from users
    interface App<ListBrand, 'A>
    member _.Value = xs

let injList (xs: 'A list) : App<ListBrand, 'A> = ListApp xs :> App<ListBrand, 'A>
let prjList (a: App<ListBrand, 'A>) : 'A list = (a :?> ListApp<'A>).Value

// The witness: what "instance Functor []" becomes
type Functor<'F> =
    abstract Map<'A, 'B> : ('A -> 'B) -> App<'F, 'A> -> App<'F, 'B>

let listFunctor =
    { new Functor<ListBrand> with
        member _.Map f xs = injList (List.map f (prjList xs)) }

// Generic code, written and compiled once
let twice (fn: Functor<'F>) (xs: App<'F, int>) : App<'F, string> =
    fn.Map string (fn.Map (fun n -> n * 2) xs)

prjList (twice listFunctor (injList [1; 2; 3]))   // ["2"; "4"; "6"]
```

The object `listFunctor` plays the role of `instance Functor []`. As in Haskell, each container needs its own instance, written once; what the encoding recovers is that `twice` is written once and works for every container with a witness (an `optionFunctor` would be defined just like `listFunctor`). `twice` is the function that was rejected in Section 2.5, and it type-checks now for two reasons. First, `App<'F,'A>` is a legal type that *mentions* `'F` without applying it. Second, `Map` is a generic method, so it remains polymorphic in `'A` and `'B` even when the witness `fn` is passed as an argument: the witness plays the role of the `Functor` dictionary that GHC passes implicitly. The price is the following:

- **Kinds by convention only.** `'F` still has kind `Type`. Nothing stops a programmer from writing `App<int, string>` or `Functor<bool>`; that `'F` names a type constructor is a convention, not a checked fact.

- **A run-time cast.** `prjList` relies on a downcast, which is safe only if every value of type `App<ListBrand,_>` was produced by `injList`. In OCaml the module system can enforce this invariant by keeping the application type abstract; in F#, `App<'F,'A>` is an ordinary interface that any code may implement, so the invariant holds only by discipline, and a violation surfaces as an `InvalidCastException` at run time rather than as a compile error.
- **Boilerplate and indirection.** Every container needs a brand, a wrapper and conversions at the boundary, and witnesses are passed explicitly where Haskell resolves instances automatically.

Computation expressions. For monads, F# offers a syntactic answer. A *computation expression* such as `async { let! x = op ... }` is translated into calls to methods of a *builder* object (`async`, with methods `Bind`, `Return`, ...), and the result is then type-checked normally. One notation thus serves `async`, `task` and user-defined builders, like Haskell's `do`, without a type for "any monad"; but each builder serves one type constructor, and a function generic over "the monad of the builder" cannot be written.

2.8 Consequences for library design

The absence of higher-kinded abstraction shapes the F# core library in a recognisable way. Each container comes with its own module: `List`, `Array`, `Seq`, `Option`, `Result`, and others. These modules offer parallel sets of functions with the same names and analogous signatures (`map`, `bind`, `filter`, `fold`, ...). The regularity is real but rests on convention. In Haskell, a single `Functor` class gives a checked interface that every container implements once, and code written against the class works for all of them. In F#, consistency between `List.map` and `Option.map` is maintained by the library designers, and code is reused by substituting one module name for another.

Where F# does abstract over containers, it uses the object-oriented mechanism available on .NET: the interface `seq<'T>` (an abbreviation of `IEnumerable<'T>`), which lists and arrays implement. `Seq.map` therefore works on all of them, but its type `('T -> 'U) -> seq<'T> -> seq<'U>` reveals what is lost: the result is a `seq`, not a list or an array. Subtyping abstracts over a *type* (`seq<'T>`, of kind `Type`), and it forgets the specific constructor. Preserving the constructor, so that mapping a list yields a list, is exactly what `f a -> f b` expresses and what requires a variable of kind `Type -> Type`.

These choices reflect the design history of the language. Higher-kinded type parameters have no counterpart in .NET generics, which was a recognised obstacle to running Haskell code on .NET when F# was created. Type classes, the mechanism through which Haskell exploits higher kinds, were deliberately left out of F#, for reasons that included their interaction with the object-oriented part of the language and the performance cost of passing dictionaries at run time. F# instead favoured mechanisms resolved at compile time, such as `inline` functions with member constraints, and syntactic ones, such as computation expressions. Libraries that want Haskell-style abstractions, such as FSharpPlus, build them from these mechanisms and accept the limitations described in Section 2.7.

2.9 Summary: F# compared with Haskell's kinds

Table 3 summarises the comparison. To answer the question posed at the beginning: F# can express the first-order part of Section 1, namely type constructors applied to types and synonyms with all their parameters, and its compiler rejects ill-formed applications of them for reasons of arity, much as Haskell does for reasons of kind. It cannot treat a type constructor as something a variable can stand for, so by the criterion of Section 1 it fails the test; its

Table 3: Haskell's kind-related features and their status in F#.

Status in F#	**Features**
Directly analogous	Generic type definitions as first-order type constructors (`option`, `list`, `Result`); type variables of kind `Type`; rejection of applications with the wrong number of arguments; abbreviations as synonyms with all parameters written out.
Similar, but not kinds	Generic arity (a number; uncurried, always saturated); constraints such as `comparison` (predicates on types, like class constraints); the type/measure distinction (two base sorts, no arrows).
Not expressible	Unapplied or partially applied type constructors, including synonyms that denote them (`type Validated = Either String`); type variables of kind `Type -> Type`; types such as `Rose` and `MaybeT`; constructor classes (`Functor`, `Monad`); type families; kind inference and polymorphism.
Partially recovered	`inline` functions with member constraints (overloading, without kind discipline); interfaces with static abstract members (classes over types only); the `App<'F,'A>` encoding with witnesses (abstraction, with conventions and casts instead of kinds); computation expressions (notation for monads, no abstraction).

workarounds each recover only part of what Haskell's kinds provide together: overloading, abstraction and static safety.

3 Approximating Kinds in TypeScript

Haskell and F# check types as part of compiling the program. JavaScript has no static types at all, so the question of Section 1 must be asked of TypeScript, the statically typed extension of JavaScript whose types are checked and then removed when the code is compiled to plain JavaScript:

> *How far can TypeScript approximate the abstractions enabled by Haskell's kinds, and what guarantees are lost when those abstractions are encoded rather than supported natively?*

The short answer has three parts. Natively, TypeScript is in the same position as F#: it has first-order type constructors, rejects some ill-formed applications of them, and has no type variables of kind `Type -> Type`. Unlike F#, however, it can *compute* types: it can look up a property type, branch on a type, and infer parts of a type. This makes possible an encoding of higher-kinded polymorphism in which application is evaluated by the compiler itself, without the wrappers and casts of Section 2.7. What is lost in the encoding is kind checking in the sense of Section 1: it is replaced by membership in a hand-maintained registry, and several guarantees that Haskell gives for free become conventions.

3.1 TypeScript's types and JavaScript's run time

TypeScript extends JavaScript's syntax with type annotations, interfaces, type aliases and generics. Its compiler, `tsc`, checks the program and emits JavaScript in which all of these have been *erased*: annotations never change what the program does at run time, and no information about types survives as run-time data. (A few TypeScript-specific constructs, such as `enum`, do generate code; none is used in this section.) Generics are written with type parameters in angle brackets:

```typescript
// TypeScript
function first<T>(xs: T[]): T | undefined {
  return xs[0];
}
const n = first([1, 2, 3]);
// n : number | undefined
```

```javascript
// Emitted JavaScript
function first(xs) {
    return xs[0];
}
const n = first([1, 2, 3]);
```

The type parameter `T` is inferred as `number` at the call, exactly as `a` would be in Haskell, and disappears from the output. Three further properties of the type system matter for what follows.

- **Structural typing.** Two types are compatible when their members are compatible, whatever their names. A type alias is just a name for a type: it introduces nothing new.
- **Deliberate unsoundness.** Some operations are accepted although their safety cannot be established statically (the type `any`, unchecked casts with `as`, bivariant method parameters). Static guarantees are stronger, though still not complete, for code that avoids `any` and `as`, with the `strict` options enabled (the default since TypeScript 6.0).
- **Checking does not gate execution.** By default `tsc` emits JavaScript even when it reports type errors. One way to avoid this is with the `noEmitOnError` option enabled.

Ordinary generics do not give higher-kinded polymorphism, for the same reason as in F#: a type parameter stands for a *type*. It can be constrained (`T extends Lengthwise` requires a member `length`, a little like a class constraint on a type of kind `Type`), but it cannot be applied to arguments. The next subsection makes this precise.

3.2 Kinds in TypeScript: what exists natively

First-order type constructors. Generic interfaces and generic type aliases play the role of Haskell's type constructors. The counterparts of `Maybe` and `Either` can be written as tagged unions, and `Array<A>` (written `A[]`) is built in:

```
type Option<A> =
  | { readonly tag: "none" }
  | { readonly tag: "some"; readonly value: A };
type Result<E, A> =
  | { readonly tag: "err"; readonly error: E }
  | { readonly tag: "ok"; readonly value: A };
type Validated<A> = Result<string, A>;   // alias with all parameters written out
```

As in F#, such a constructor appears in a type only when applied to all its arguments, and the compiler rejects applications with the wrong number of arguments, applications of non-generic types and applications of type parameters:

```
let ys: Array = [];                          // TS2314
type V = Result<string>;                     // TS2314
interface Point { readonly x: number }
type P = Point<number>;                      // TS2315
function g<F>(x: F<number>): F<number> { return x; }   // TS2315
```

Both errors are about generic arity: TS2314 reports that a generic type has received the wrong number of type arguments, and TS2315 that type arguments were given to something that is not generic. The errors on `ys` and `P` correspond to the first two kind checks of Section 1.5 and have the same practical effect, exactly as in Section 2.3. The error on `V` is different in nature: the corresponding Haskell synonym `type V = Either String` is legal, so this line shows the absence of partial application rather than a kind error. The error on `g` is the decisive one: like FS0712 in F#, TS2315 rejects a type variable used as a type constructor, so `f a` cannot be written and neither can `Functor`. The alias `Validated<A>` is the counterpart of an F# abbreviation or of a Haskell synonym with all its parameters written out; there is no way to write `Either String` as a type expression on its own. Default type arguments (`<A = string>`) do not change this: they fill in missing arguments, but the result is still a fully applied type.

Type-level code. Where TypeScript goes beyond F# is in computing types. Three constructs matter here: the first two are the building blocks of the encoding of Section 3.3, the third gives a counterpart of the type family `Elem` of Section 1.6.

- An *indexed access type* `T[K]` is property access at the type level: it is the type of the property `K` of the object type `T`. The key `K` is itself a type, normally a string literal type such as `"age"`; if `K` is a union of keys, `T[K]` is the union of the corresponding property types.
- `keyof T` is the union of the property names of `T`, each as a string literal type.
- A *conditional type* `C extends P ? X : Y` is `X` if `C` is assignable to `P`, and `Y` otherwise. Inside `P`, `infer E` introduces a variable `E` that is bound to the matching part of `C` and can be used in `X`.

```
interface Person { readonly name: string; readonly age: number }
type AgeType = Person["age"];                // number
type Keys = keyof Person;                    // "name" | "age"
type AnyField = Person[keyof Person];        // string | number
type ArrayElem<C> = C extends readonly (infer E)[] ? E : never;
```

```
type B = ArrayElem<boolean[]>;              // boolean
```

Combining two conditional types gives a close counterpart of `Elem`:

```
type Elem<C> = C extends readonly (infer E)[] ? E
             : C extends ReadonlySet<infer E> ? E
             : never;
type E1 = Elem<boolean[]>;                  // boolean
type E2 = Elem<Set<string>>;                // string
```

Such type-level functions, however, take and return types of kind `Type`, and are themselves generic aliases that must be applied to all their arguments: `Elem` cannot be passed as an argument any more than `Array` can. Nor are they kind-checked in the sense of Section 1.6: a conditional type is checked by assignability, and an argument outside the intended domain simply selects the `never` branch instead of being rejected. Table 4 summarises the native situation.

Table 4: Concepts of Section 1 in TypeScript, without encodings.

Concept	**TypeScript**
Type constructors	Generic interfaces and aliases (`Array<A>`, `Option<A>`); first order, always fully applied
Kind checks 1 and 2	Arity errors TS2314 and TS2315, with the same effect, as in F#
Partial application	None; only aliases with all parameters (`Validated<A>`)
Higher-kinded variables	None: `F<A>` with a type parameter `F` is rejected (TS2315)
`Functor`, kind check 3	Not expressible; only through an encoding (Section 3.3)
Type-level code	Indexed access, `keyof`, conditional types: functions on types of kind `Type`, checked by assignability
Kind inference, kind polymorphism	None

3.3 A URI-based encoding of Functor

The missing ingredient is a way to write "`F` applied to `A`" when `F` is a variable. TypeScript has no feature for this; the usual technique is a programming pattern, the defunctionalisation already met in Section 2.7 (lightweight higher-kinded polymorphism): a type constructor is represented by a *name*, and application by a table that maps names to types. In TypeScript the name is a string literal type, called the constructor's *URI*, and the table is an interface. The code below is a simplified version of the design of fp-ts, an independent open-source library for functional programming in TypeScript, whose development has since merged into the Effect ecosystem (Section 3.4).

```
interface URItoKind<A> {}                          // the registry
type URIS = keyof URItoKind<unknown>;              // registered names
type Kind<F extends URIS, A> = URItoKind<A>[F];    // "F applied to A"

interface Functor<F extends URIS> {
  readonly URI: F;
  readonly map: <A, B>(fa: Kind<F, A>, f: (a: A) => B) => Kind<F, B>;
}
```

Each component has a precise role.

- `URItoKind<A>` is the registry. It is never used as the type of a value; each of its properties will associate a URI (the property name) with the corresponding constructor applied to `A` (the property type). Together with the URIs it plays the role of the brand types of Section 2.7, with one difference: here the table is evaluated by the compiler.
- `URIS` is the union of the registered names. It plays the role of a kind: `F extends URIS` reads "`F` is one of the registered one-parameter constructors". The argument `unknown` is only there to instantiate the generic interface before taking its keys.
- `Kind<F, A>` is type application, implemented as an indexed access. When `F` is a concrete URI it reduces to an ordinary type; when `F` is a type parameter it remains unevaluated, an abstract "`F` of `A`".
- `Functor<F>` is the counterpart of the class `Functor`, represented, as in Section 2.7, by the type of an explicit dictionary. Its `map` is generic in `A` and `B`, so it remains polymorphic when the dictionary is passed as an argument. The field `URI`, kept from fp-ts, records the constructor in the dictionary value itself. In the instances below `F` is already fixed by their declared types (`Functor<"Array">`); the field matters when a dictionary is written without such an annotation, because the compiler can then infer `F` from the type of `URI`.

Registering constructors and writing instances. A constructor becomes available by adding a property to the registry. TypeScript merges all declarations of an interface with the same name into one, so registration is just another declaration of `URItoKind`; in a program split into modules, a module registers its constructor with a *module augmentation* (`declare module "..." { interface URItoKind<A> { ... } }`). Instances are ordinary objects:

```
interface URItoKind<A> {
  readonly Array: Array<A>;
  readonly Option: Option<A>;
}

const none: Option<never> = { tag: "none" };
const some = <A>(value: A): Option<A> => ({ tag: "some", value });

const arrayFunctor: Functor<"Array"> = {
  URI: "Array",
  map: (fa, f) => fa.map((a) => f(a)),
};
const optionFunctor: Functor<"Option"> = {
  URI: "Option",
  map: (fa, f) => (fa.tag === "some" ? some(f(fa.value)) : none),
};
```

In `arrayFunctor`, the parameter `fa` has type `Kind<"Array", A>`, which the compiler reduces to `A[]`; this is why `fa.map` is accepted. No conversion between an encoded and a real representation is needed, because `Kind<"Array", A>` *is* `A[]`. Generic code is then written once, against the dictionary:

```
function twice<F extends URIS>(
  functor: Functor<F>, fa: Kind<F, number>
): Kind<F, string> {
  return functor.map(functor.map(fa, (n) => n * 2), (n) => String(n));
}
const xs = twice(arrayFunctor, [1, 2, 3]);     // ["2", "4", "6"] : string[]
```

```
const ox = twice(optionFunctor, some(21));     // some("42")      : Option<string>
```

This is the function `twice :: Functor f => f Int -> f String` that F# could not write directly Section 2.5 and wrote via `App` in Section 2.7. The type parameter `F` names the constructor, while the ordinary parameter `functor` is the dictionary. At the calls, `F` is inferred from the type of the dictionary (`"Array"` or `"Option"`), the second argument is checked against `number[]` or `Option<number>`, and the results receive the concrete types shown.

What the compiler checks. The encoding turns several kind errors into ordinary type errors:

```
type L = Kind<"List", number>;              // TS2344
type K = Kind<"Array", number, string>;     // TS2314
const badFunctor: Functor<"Option"> = {
  URI: "Option",
  map: (fa, f) => [],                       // TS2322
};
function fake<F extends URIS>(): Kind<F, number> { return [1]; }   // TS2322
interface URItoKind<A> { readonly Array: Set<A> }      // TS2717
```

An unregistered name is rejected because it does not satisfy the constraint `URIS` (TS2344), which is the encoded counterpart of kind check 3; `Kind` must receive exactly two arguments; an instance must transform the constructor it is registered for (here, `never[]` is not an `Option<B>`); and two registrations of the same name with different types conflict. Finally, generic code cannot fabricate a value of the abstract type: in `fake`, the array `[1]` is not accepted as a `Kind<F, number>`, because `F` might be `"Option"`. A generic function such as `twice` can obtain such values only from its caller or through the dictionary.

What remains a convention. Other guarantees are not provided, and the compiler accepts all of the following:

```
interface URItoKind<A> { readonly Num: number }        // ignores A
const numFunctor: Functor<"Num"> = {       // accepted
  URI: "Num", map: (fa, f) => fa,
};

function size<F extends URIS>(functor: Functor<F>, fa: Kind<F, number>): number {
  return Array.isArray(fa) ? fa.length : 1;  // accepted: inspects which F
}

declare function mapK<F extends URIS, A, B>(
  fa: Kind<F, A>, f: (a: A) => B): Kind<F, B>;
const ws = mapK([1, 2], (k) => k + 1);      // TS18046: k is of type unknown
```

- **The registry does not know what a constructor is.** Any property type is accepted. `Num` ignores its argument, so the analogue of `instance Functor Int`, rejected by Haskell's kinds, is accepted here and silently behaves as a constant functor (`Const Int` in Haskell terms). That registered names denote constructors of kind `Type -> Type` is a convention.
- **No parametricity.** For a type parameter `F`, the compiler relates `Kind<F, number>` to the union of all registered types (`number[] | Option<number> | number` here), so generic code may test at run time which container it was given, as `size` does. A Haskell function of type `Functor f => f Int -> Int` cannot inspect `f` (without extra classes such as `Typeable`).

- **No instance resolution.** `F` can be inferred from a dictionary, but not from a container: an indexed access cannot be inverted to recover `"Array"` from `number[]`. In `mapK`, `F` falls back to its constraint and the element type to `unknown`. Dictionaries must therefore be passed explicitly, nothing prevents several different `Functor<"Array">` objects, and the choice between them is the programmer's, whereas Haskell infers `f` by unification and finds the unique instance.
- **No laws.** As in Haskell, the functor laws are not checked.

3.4 What the encoding cannot express

One registry per kind. The registry fixes the shape of its constructors: every entry takes exactly one argument `A`, so `URIS` amounts to the single kind `Type -> Type`. A two-parameter constructor such as `Result<E, A>` needs a second registry with two parameters, its own `URIS2` and `Kind2`, and its own versions of `Functor` and of every function written against it; fp-ts maintains four such registries, for one to four parameters. Kinds thus become a fixed menu of separate encodings. There is no kind variable, no kind polymorphism, no kind inference, and no kind of the form `(Type -> Type) -> Type`: a monad transformer such as `MaybeT` of Section 1.4 would require yet another encoding layer on top of this one.

Higher-kinded data types. Within that menu, the encoding is genuinely expressive. A data type parameterised by a registered constructor, the rose tree of Section 1.4, is written directly:

```
interface Rose<F extends URIS, A> {
  readonly value: A;
  readonly children: Kind<F, Rose<F, A>>;
}
const tree: Rose<"Array", number> =
  { value: 1, children: [{ value: 2, children: [] }] };
const path: Rose<"Option", number> =
  { value: 1, children: some({ value: 2, children: none }) };
```

Because `Kind<"Array", Rose<"Array", number>>` reduces to an ordinary array type, the values are plain object and array literals.

Partial application. The registry cannot express `Either String`, a constructor obtained by fixing one argument of another. With a single registry there are two options. The first is to register each partial application under its own name, which fixes the argument once and for all:

```
interface URItoKind<A> { readonly Validated: Result<string, A> }
const validatedFunctor: Functor<"Validated"> = {
  URI: "Validated",
  map: (fa, f) => (fa.tag === "ok" ? { tag: "ok", value: f(fa.value) } : fa),
};
const vs = twice(validatedFunctor, { tag: "ok", value: 5 });
// vs : Result<string, string>
```

The second, used by more recent libraries such as Effect, replaces the registry by *type lambdas*: an interface whose member `type` mentions `this["A"]`, and an application operator that fixes `A` by intersection.

```
interface TypeLambda { readonly A: unknown; readonly type: unknown }
type Apply<F extends TypeLambda, A> = (F & { readonly A: A })["type"];
```

```ts
interface ArrayL extends TypeLambda { readonly type: Array<this["A"]> }
interface ResultL<E> extends TypeLambda { readonly type: Result<E, this["A"]> }

type T1 = Apply<ArrayL, number>;            // number[]
type T2 = Apply<ResultL<string>, number>;   // Result<string, number>
```

`ResultL<E>` behaves like the partial application `Either e`: it is a one-parameter constructor obtained from a two-parameter one, and the fixed argument may itself be a type parameter, which no registry entry can provide. This variant needs no central registry, but it shares the other weaknesses of the encoding: `Apply` accepts any interface that extends `TypeLambda`, including one whose `type` ignores `A`, instances are still explicit dictionaries, and the resulting types are harder to read.

Costs. In practice, the encoding has a price that native kinds do not:

- **Boilerplate.** Every constructor must be registered, every instance carries a `URI`, and each additional kind duplicates the whole apparatus.
- **A global, open registry.** Any module can augment `URItoKind`. A clash is reported only when two registrations of the same name have different types (TS2717); identical registrations merge silently, and the registry, not the kind system, decides what may instantiate `F`.
- **Diagnostics in encoded terms.** Errors mention `keyof URItoKind<unknown>` or indexed access types instead of kinds, and the reader must translate them back.

3.5 Practical value and comparison with F#

The benefit is the one identified in Section 1.7: generic code such as `twice`, and more realistically a library of operations defined once for every functor, applicative or monad, is written once and checked once, and each new container only has to provide a dictionary. This is how fp-ts and Effect organise their type classes. Natively, TypeScript code instead follows the pattern of Section 2.8: methods such as `Array.prototype.map` are defined separately for each container.

Table 5 compares the encoding with Haskell and with the two F# workarounds of Section 2.7: the `App<'F,'A>` encoding with witnesses, and `inline` functions with member constraints.

Table 5: Higher-kinded abstraction in Haskell, F# and TypeScript.

	Haskell	**F# encoding**	**F# inline**	**TypeScript**
"`f` of `a`"	`f a`	`App<'F,'A>`	`^Fa`, `^Fb`	`Kind<F, A>`
Application computed	native	no	—	yes
Run-time conversions	none	wrapper, downcast	none	none
Instance selection	inferred	explicit	overloading	explicit
Discipline on `f`	kinds	convention	none	registry
Partial application	leading arguments	none	none	by registration
Generic code	once	once	inlined per use	once

The main gain over the F# encoding comes from type-level computation. In F#, the encoded type `App<ListBrand,'A>` differs from the real type `'A list`, so every value crosses the boundary through `inj` and `prj`, and the projection is a run-time downcast that could fail. In TypeScript, `Kind<"Array", A>` *reduces* to `A[]`: there is no wrapper, no cast, and no way for

the projection to go wrong. The registry also gives more discipline than F#'s brands, whose use is a pure convention: an unregistered name is rejected. Compared with F#'s inline functions and member constraints, the TypeScript encoding keeps the relationship between `Kind<F, A>` and `Kind<F, B>` visible in the type of `map`, and produces ordinary generic functions that are not inlined; but it loses overload resolution, so the dictionary is passed by hand. On the other side, F# has a nominal type system whose generics exist at run time and whose downcasts are checked when they execute, whereas TypeScript's guarantees are structural, can be bypassed with `any` or `as`, and vanish entirely after compilation. None of the three approaches checks that a type is a constructor in the sense of Section 1: in each, that remains a convention.

3.6 Compilation and run time

Compiling the encoding shows exactly what it costs at run time.

```
const arrayFunctor = {
    URI: "Array",
    map: (fa, f) => fa.map((a) => f(a)),
};
const optionFunctor = {
    URI: "Option",
    map: (fa, f) => (fa.tag === "some" ? some(f(fa.value)) : none),
};
function twice(functor, fa) {
    return functor.map(functor.map(fa, (n) => n * 2), (n) => String(n));
}
```

Everything that encodes kinds (`URItoKind` and its registrations, `URIS`, `Kind`, the interface `Functor`, the type parameters and all annotations) has disappeared. What remains is ordinary JavaScript: containers are plain arrays and objects, instances are plain objects, and generic code receives its dictionary as an ordinary argument. The only run-time machinery introduced by the encoding is therefore this explicit dictionary passing, essentially what a Haskell compiler does implicitly for class constraints, and the `URI` string, which serves the type checker and is never read at run time.

Nothing checks kinds, or even types, when the program runs. If the program contains the call `twice(arrayFunctor, some(21))`, `tsc` reports TS2345 (an `Option<number>` is not a `number[]`) but, by default, still emits JavaScript, which then fails with "`fa.map is not a function`". Run-time protection would require explicit checks written by the programmer, such as the tag tests already present in `optionFunctor`.

Without TypeScript, plain JavaScript can approximate constructor classes only by run-time convention. The Fantasy Land specification, for instance, calls a value a `Functor` when it has a method named `fantasy-land/map` that obeys the functor laws. Such conventions make libraries interoperable, but nothing checks them: they are an agreement about method names, not a kind system.

Summary. To return to the question posed at the beginning: TypeScript can approximate higher-kinded polymorphism more closely than F#, because type application can be encoded as type-level computation that the compiler evaluates, so that `Functor`, `twice` and even `Rose` can be written with no run-time conversion. What is lost is everything that Haskell derives from kinds themselves. Ill-formedness is detected only as far as the registry and the arity of `Kind` reach; a registered name need not denote a constructor; generic code is not parametric in `F`; instances are neither inferred nor unique; partial application must be anticipated, by registration or by type lambdas; and every guarantee ends at compilation, leaving JavaScript that runs with no knowledge of any of them.

4 Conclusion

The comparison was guided by the question with which Section 1 closed: can a language express a variable of kind `Type -> Type`? Haskell answers it natively, and this is what makes `Functor`, `Monad`, `Rose` and monad transformers ordinary, statically checked definitions. F# and TypeScript answer it negatively, and in strikingly similar terms: their generic types are first-order constructors, always fully applied, and both compilers reject a type variable at the head of an application (FS0712, TS2315). What survives of Haskell's kind system is its first-order fragment, reduced to checking generic arity; everything that depends on arrow kinds, from partial application to constructor classes, is absent.

The languages differ in how much of the missing expressiveness can be recovered. Both rely on the same idea, lightweight higher-kinded polymorphism, originally devised for OCaml: a constructor is represented by a name, its application by an ordinary type, and instances by explicit dictionaries. In F#, the encoded application is distinct from the real type, so values cross the boundary through wrappers and a downcast whose safety rests on discipline; inline functions with member constraints recover overloading instead, without relating input and output constructors. In TypeScript, type-level computation lets the compiler evaluate the encoded application itself, so `Kind<"Array", A>` *is* `A[]`, and the approximation is correspondingly closer. Plain JavaScript is left with conventions on method names.

The trade-off is the same in every case. Encodings recover abstraction, generic code such as `twice` written once for every container, but never the discipline that kinds provide: a registry entry that ignores its argument, `Functor<bool>` or a `map` over a `string` are all accepted, partial application must be anticipated, instances are passed by hand rather than inferred, and diagnostics speak the language of the encoding. Native support delivers these guarantees together; each encoding buys part of the expressiveness by replacing them with conventions, boilerplate and, in F#, run-time casts.

Kinds, then, are more than a classification of type expressions: they are what makes abstraction over type constructors both expressible and safe. Languages without them can approximate the abstractions that kinds enable, but not the guarantees that come with them. The criterion of Section 1 separates Haskell from F# and TypeScript not by which programs can eventually be written, but by whether the compiler can verify that abstraction over type constructors is used correctly.